ASSIGNMENT #01

Prof. Bilal Khan

Object Oriented Programming (CS-1004)

SE-2A | CS-2A | CS-2B | CS-2C

Instructions:

- **Plagiarized work will be given zero marks.**
- **Late submissions will not be entertained.**
- All assignments should be done individually.
- Add appropriate comments in your code for assumptions.
- Submit your .cpp files in GCR, In the given naming format below;

P25-1234_Name_Section_Q1.cpp

Rules

- No vectors, lists, maps, or classes
- No fixed-size arrays
- Must resize arrays manually
- Must make sure proper memory deallocation.

The Shape Shifting Hostel

Every semester, FAST hostel changes its room sizes.
Some rooms suddenly get new students, others lose them overnight.

The warden wants a program that resizes itself without crashing.

Task

Design a C++ program;

Having a **STRUCT** named Student with attributes roll No. and name of the student and another **STRUCT** named Room with attributes room No, total capacity, No. of current Students and an array of the struct Student.

1. Dynamically create N rooms.
2. For each room, dynamically create a student list.
3. Implement: (Must have separate functions for each of the following)
    - Add a new room (resize Room array)
    - Add student to a room
    - Remove student from a room
    - Display all rooms with students
    - Find the most crowded room
    - If a room is full, move the student to the nearest room that has space. (Optional but I'll reward you extra marks for this)

The Game Inventory That Eats Memory

You're building a game for a small studio.

Players collect mystic items, but the game has a bug:
items sometimes vanish, duplicate, or turn into ghosts because of bad memory handling.

Your job is to create a safe inventory system that never leaves ghost items behind.

Task

Design a menu-driven C++ program having a **STRUCT** named Item with attributes id, name and power.

The inventory must:

1. Start with a dynamic array of size 3.
2. Allow: (Must have separate functions for each of the following)
    - Add new item (resize array +1)
    - Delete item by ID (shrink array)
    - Upgrade item power
    - Display all items
3. Prevent ghost items:
    - After deletion, no unused memory should remain.
    - Inventory must always be resized correctly.

Twist

If an item's power goes above 100, it becomes "cursed":

- It is automatically deleted from memory.
- A warning is displayed.

```
Item has been cursed and removed from memory!
```

LIBRARY MANAGEMENT SYSTEM

FAST Library is facing a strange issue.

Bookshelves expand during exams…
They shrink after semester ends…
Sometimes books disappear mysteriously.

The librarian wants a **memory-safe system** that dynamically adjusts shelves and books without leaking memory.

If memory is mishandled, the system “forgets” books permanently.

You are hired to prevent that disaster.

TASK

Design a **menu-driven C++ program** using:

- A Structure name **Book**
- A Structure name **Shelf**

Book Structure:

- int id
- char title[50]
- float price

Shelf Structure:

- int shelfNo
- int capacity
- int currentBooks
- Book* books

Menu example and the functions it should include:

```
========== FAST LIBRARY SYSTEM ==========
1. Create Initial Shelves
2. Add New Shelf (Resize Shelf Array)
3. Add Book to Shelf
4. Remove Book by ID (Shrink Shelf Books)
5. Display All Shelves and Books
6. Find Most Expensive Book
7. Merge Two Shelves
8. Remove Empty Shelves Automatically
9. Exit
```

(Must be implemented in separate functions)

1. Add new shelf:

- Resize the Shelf array (Increase it by one)
- Take shelf number and capacity
- Allocate book array dynamically

2. Add Book:

- Ask for shelf number
- If space available → add book
- If shelf is full → resize its book array manually (+1 capacity)

3. Remove Book by ID:

- Locate the shelf
- Remove the book
- Shrink the book array size
- Properly deallocate unused memory

4. Display shelf:

Display:

- Shelf number
- Capacity
- Current books
- All book details

5. Find most expensive book:
Traverse all shelves and books
Display book with highest price and its shelf

6. Merge two shelves:

- User selects two shelf numbers
- Move all books of second shelf into first shelf
- Resize first shelf accordingly
- Delete second shelf
- Resize shelf array (-1)

7. Remove shelf automatically:

If any shelf has zero books do the following:

- Deallocate its book array
- Remove shelf from system
- Resize shelf array

8. Exiting the program:

Before exiting:

- Deallocate all books
- Deallocate all shelves